INDEPENDENT SECURITY REVIEW

2factor

Smart Contract Security Assessment

Repository commit
`2e527dac9890dd8a857edeae57285d09fdef4375`

Audit period (UTC)
2026-08-28T13:58:46Z — 2026-08-28T14:49:59Z

Report generated
2026-08-28T15:20:52Z

Lead model
Not exposed by runtime

Reasoning / effort
Not exposed by runtime

Report version
1.0.1

CONFIDENTIAL · FINAL

Table of Contents

Document Control 3
Version history . . . 3
Engagement record . . . 3
Execution provenance . . . 3
Execution durations . . . 3
Executive Summary 4
Severity summary . . . 4
Scope and Methodology 4
Scope assumption . . . 4
Method . . . 5
Test strategies . . . 5
Architecture and Trust Model 5
System overview . . . 6
Asset flow . . . 6
Upgrade and initialization model . . . 6
Oracle model . . . 6
Actors and trust boundaries . . . 7
Security Properties Tested 8
Findings Summary 9
Detailed Findings 9
INFO-04 (formerly MED-01) — Permissionless settlement uses the first poke's post-eligibility price . . . 9
LOW-02 (formerly MED-02) — Redemption without upkeep leaves the caller's resting-underlying share behind . . . 10
LOW-01 — Stateless routers expose ambient balances and persistent approvals to arbitrary callers and prior spenders . . . 10
Economic and Design Risks 11
Privileged authority . . . 11
Token assumptions . . . 11
Oracle, liquidity, and liveness . . . 11
Permissionless maintenance . . . 11
MEV and ordering . . . 11
Centralization and Privileged Operations 12
Testing Results 12
Baseline . . . 12
Fuzz and invariant campaign . . . 12
Coverage . . . 13
Negative results . . . 13
Informational Observations 13
Conclusion 13
Appendix A — Complete File Inventory 14
Appendix B — Public Function and Role Inventory 17
Appendix C — Candidate Disposition 25
Appendix D — Tools and Commands 26
Appendix E — Glossary and Disclaimer 27

Document Control

Version history

Version	Generated UTC	Status	Description
1.0.1	2026-08-28T15:20:52Z	Final	Incorporated developer responses; reclassified MED-01 to INFO-04 and MED-02 to LOW-02.
1.0.0	2026-08-28T14:50:31Z	Superseded	Initial independent assessment of commit 2e527da.

Engagement record

Field	Value
Protocol	2factor
Repository	`git@github.com:fragmentsorg/2factor.git`; local path `/Users/nithin/projects/ampl-core/2ff`
Commit	`2e527dac9890dd8a857edeae57285d09fdef4375`
In scope	All Solidity below `core-contracts/contracts/` and `vaults/contracts/`; related deployment modules, tests, inherited behavior, and interfaces as supporting material
Out of scope	Subgraph and off-chain ops logic, front ends, production deployments, live economics, unprovided infrastructure, and source of external protocols
Compiler	Solidity 0.8.35; optimizer 800; via-IR; Cancun EVM target
Build/test	Hardhat 2.29.0, Yarn 4.9.1, Node 22.15.0
Networks	No authorized public deployment supplied; local Hardhat network only
Report status	Final; no maintainer fixes or responses were supplied

Execution provenance

Execution ID	Agent role	Exact model	Effort	Started UTC	Completed UTC	Purpose
lead-01	lead auditor	Not exposed by runtime	Not exposed by runtime	2026-08-28T13:58:46Z	2026-08-28T14:49:59Z	System reconstruction, manual review, adversarial testing, findings validation, and report delivery

Execution durations

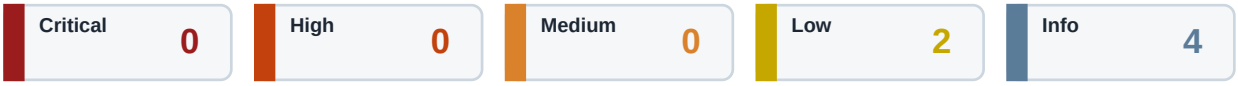
Measure	Duration	Basis
Wall-clock audit	3,073 seconds	UTC audit start through completion
Active analyst execution	Not exposed by runtime	No reliable active-time metric was available; no estimate was substituted
Testing, fuzzing, and coverage	1,998 seconds	Sum of rounded suite-reported durations; parallel overlap not removed
Final PDF build	1 second	Timed final deterministic ReportLab build, rounded to whole seconds

Runtime metadata did not expose an exact model identifier, effort setting, or agent-runtime version. Those values are reported verbatim as unavailable and were not inferred from product branding. No auxiliary model executions were used. Compiler, linter, coverage, test, and rendering processes are tools rather than independent reviewers.

Model-assisted auditing is probabilistic and does not guarantee that the system is free of vulnerabilities. This report describes the reviewed commit and tested assumptions; later code, configuration, deployment, governance, markets,

and external dependencies can change the risk.

Executive Summary



2factor segments the economic exposure of an underlying asset into senior and junior bond tranches, aggregates those tranches into senior and junior perpetual tokens, and offers vaults that package the perps into underlying- or USD-denominated positions. Permissionless maintenance rolls aging tranches, rebalances senior and junior reserve value, issues new bonds, trades through configured DEX strategies, and can settle passive CoW orders.

The assessment reviewed 11,001 lines of production Solidity across the core and vault packages, plus deployment topology, malicious mocks, interfaces, and relevant inherited behavior. The repository compiled and its baseline tests passed under Node 22. A deterministic campaign exercised 81 fuzz and invariant properties with a fixed seed. Coverage runs reached 99% or more lines in both contract packages, while manual review traced value-moving entry points, approvals, callbacks, oracle reads, proxy administration, and privileged parameter changes.

No Critical, High, or Medium issue and no direct path to steal protocol-custodied funds under the intended standard-token and honest-administrator assumptions was confirmed after developer response. Core conservation, reserve backing, proportional share accounting for enumerated assets, fee caps, rollover value balance, CoW order floors, and common reentrancy boundaries held in the executed suites.

After developer response, no Medium issue remains. The developers confirmed that a bond's maturity date is intentionally the earliest time at which a permissionless poke may settle it, and that the oracle price in that transaction intentionally determines the split. The observed timing optionality remains factual but is reclassified from MED-01 to INFO-04 because it is part of the specified instrument design. The developers also stated that redeemers are naturally incentivized to run upkeep before exiting the USD senior vault. That mitigation materially reduces likelihood but does not eliminate the discounted direct-redemption path or upkeep liveness dependencies, so MED-02 is reclassified to LOW-02. LOW-01 remains unchanged.

The revised result contains no Critical, High, or Medium finding. Before deployment or upgrade, clearly disclose INFO-04's settlement semantics, make LOW-02's discounted exit explicit and difficult to invoke accidentally, harden router balance/approval isolation, publish and verify production governance controls, and validate production oracle/DEX parameters. This assessment remains a time-bounded review rather than a production certification.

Severity summary

Severity	Count
Critical	0
High	0
Medium	0
Low	2
Informational / design	4

Scope and Methodology

Scope assumption

The engagement placeholders did not identify a scope. The audit therefore treated every Solidity file under `core-contracts/contracts/` and `vaults/contracts/` as in scope. Interfaces, external proxy import sentinels, test-only contracts, deployment modules, Hardhat configuration, and package manifests were reviewed as needed to interpret the production code. The complete per-file disposition is in Appendix A and `file-inventory.csv`.

The subgraph, Go and TypeScript operations services, automation, deployment secrets, CI badge logic, and front ends were excluded from source audit. Their tests were run where the root scripts invoked them. No production address, network, RPC, account, budget, or fork block was supplied, so public-chain interaction would have been unauthorized and was not attempted.

Method

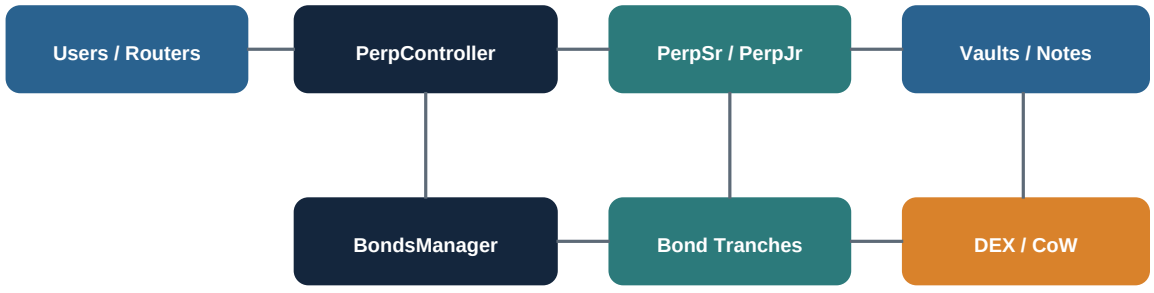
Work proceeded through repository baseline, system reconstruction, invariant definition, line review, adversarial hypothesis testing, deterministic fuzzing, coverage, candidate falsification, and report validation. Review emphasized accounting units, rounding, custody, lifecycle transitions, permission surfaces, oracle timing, DEX settlement, persistent allowances, callbacks, pauses, and worst-case administrator impact.

Automated evidence included Solidity compilation diagnostics, TypeScript/ESLint and Solidity/Solhint lint, Go vet/tests, Hardhat unit and integration suites, deterministic fuzz suites, and Solidity Coverage. Slither, Semgrep, and symbolic execution were not present locally and were not downloaded. Scanner absence is a limitation, not an implied clean result.

Test strategies

- Multi-holder bond creation, deposits, immature and mature redemption, oracle failures, extreme prices, and pooled collateral reconciliation.
- Senior/junior perp mint, redeem, flash mint/redeem, rollover, rebalance, fee settlement, reserve limits, and value conservation.
- Vault seed protection, share proportionality, donations, minimum liquidity, reentrancy mocks, repeated maintenance, fee erosion, decimals, pausing, and strategy changes.
- DEX quote/trade consistency, reference-price and health gates, Curve pool orientation and EMA checks, Uniswap v4 callback authorization, and fill sizing.
- CoW ERC-1271 order binding, token pair/direction, receiver, deadline, floor, flash-loan callbacks, relayer allowance, atomic failure, and session budget.
- Candidate-specific tests for delayed settlement pricing, resting-underlying redemption, router donations, and retained allowances.

Architecture and Trust Model



Oracle prices feed bond settlement, controller valuation, and vault health gates.

Upgrade and owner roles sit above every proxied or configurable component.

System overview

`BondsManager` escrows whitelisted underlying and deploys paired `Tranche` tokens for each bond. Deposits mint senior and junior tranche notional at the bond's opening price. At settlement, fixed collateral is divided between tranche claims. `BondIssuer` creates periodic series and tracks active bonds.

`PerpSr` and `PerpJr` are upgradeable reserve-bearing tokens controlled by `PerpController`. User and flash operations mint or burn perps against underlying or tranche reserves, with `FeePolicy` computing base, impact, flash, protocol, and AUM fees. `PerpBalancer` computes value-neutral rollover plans and system rebalance transfers. Cached oracle reads keep one price within an atomic controller or vault flow.

`UnderlyingPerpVault` packages underlying plus either perp into transferable notes. `UsdPerpSrVault` packages USD plus senior perp and uses active DEX execution or allowlisted passive swaps to track a target deviation ratio. Router contracts offer single-asset entry and exit. The CoW broker creates ERC-1271 orders, obtains just-in-time inventory through a pinned Morpho-style flash lender, and settles surplus back to the vault.

Asset flow

1. Users deposit underlying into a bond and receive paired tranches; collateral remains pooled in `BondsManager`.
2. Users or routers mint perps from tranches or underlying through `PerpController`; reserves move to `PerpSr`/`PerpJr` and fee balances remain at the controller until settlement.
3. Rollover moves stale senior value through junior liquidity into fresh bonds. Rebalance transfers reserve value from the overweight perp to the underweight perp.
4. Vault depositors supply underlying+perp or USD+perp and receive proportional notes. Vault maintenance may flash mint/redeem a perp and trade through a configured strategy.
5. Redemptions burn notes/perps/tranches and return enumerated reserve assets. Routers convert the multi-asset result to a single asset and forward balances to the caller.

Accounting can diverge from token balances at explicit boundaries: fee-on-transfer or rebasing tokens, unsolicited router balances, resting underlying in `UsdPerpSrVault`, pooled bond collateral, and external strategy execution. These boundaries drive the confirmed findings and design risks.

Upgrade and initialization model

Core components and both vault types are deployed behind OpenZeppelin TransparentUpgradeableProxy instances. A shared ProxyAdmin owner can upgrade implementations. Constructors disable implementation initializers; proxy initialization wires dependencies and assigns the deploying caller as component owner. Perp controller addresses are later assigned to perp tokens, and controller allowances are primed. Storage-layout compatibility for hypothetical future implementations was not assessable because no upgrade candidate was supplied.

Oracle model

Bond and controller pricing comes from configured `IPriceOracle` implementations. Chainlink wrappers reject non-positive answers, incomplete rounds, stale updates, and—on L2—the sequencer-down/grace-period state. `CachedOracle` uses transient storage so a flow observes one price within a transaction. The USD vault compares its strategy reference price to the controller oracle and can additionally gate a USD peg feed. Curve strategies use EMA/reference data and health checks; Uniswap v4 reference pricing follows spot and relies on the vault's cross-check.

The code validates current oracle data but does not provide a historical observation at a bond's scheduled maturity. The developers confirmed that the timestamp is intentionally an eligibility boundary and the first successful poke's current price is the settlement observation; INFO-04 records the resulting timing semantics.

Actors and trust boundaries

Actor	Code-enforced limits	Assumed / external behavior	Worst-case impact
User / MEV searcher	Slippage, ownership, balance, fee, timing, reserve, and health checks	May reorder, delay, donate, split, or atomically compose calls	Bond payout race; router ambient balance capture; bounded maintenance griefing
ProxyAdmin owner	Only proxy admin can upgrade	Secure key, review, timelock/multisig not shown	Replace logic and seize or freeze all funds under upgraded proxies
Component owner	`onlyOwner`; two-step ownership on core components	Correct oracle, issuer, fee, strategy, and limits	Redirect economics, disable flows, authorize malicious dependencies
Keeper	Dedicated modifiers	Timely and honest pause/value-cap operation	Pause user/system/vault flows; controller keeper can change value cap
Issuer	BondsManager allowlist	Creates economically sound series	Create arbitrary allowed-asset series within manager validation
Vault swapper	Owner-managed allowlist plus price/cap checks	Brings valid passive inventory	Trade only within live floor and session/cap checks; can influence resting balances
Oracle operator	Freshness/answer checks	Accurate and available feed, appropriate heartbeat	Misprice mint/redemption/settlement or halt oracle-gated operations
DEX / pool / CoW / lender	Pinned addresses and post-trade floors where applicable	Correct external code, liquidity, callbacks, and finality	Revert/liveness loss; compromised trusted addresses can exploit allowances or callbacks within gates
Token issuer	SafeERC20 call semantics	No tax, rebase, callback accounting, blacklist, or hostile upgrade	Accounting drift, freeze, or insolvency if a nonstandard token is configured

Security Properties Tested

ID	Property	Relevant components	Method	Result / limitation
INV-01	Bond senior claim plus junior claim never exceeds escrowed collateral	BondsManager	Unit + 50-run fuzz	Passed; intended settlement timing is documented in INFO-04
INV-02	Multi-bond pooled collateral covers all outstanding claims	BondsManager	Multi-bond deterministic tests	Passed for modeled standard ERC-20
INV-03	Deposit preview equals tranche mint amounts	BondsManager	Differential fuzz	Passed
INV-04	Mature and immature redemption cannot exceed caller claim	BondsManager, Tranche	Unit + multi-holder tests	Passed
INV-05	Perp reserve value and supply change proportionally under mint/redeem	PerpController, PerpTranche	Unit + fuzz	Passed within documented rounding
INV-06	Rollover preserves senior and junior value independently	PerpController, PerpBalancer	On-chain fuzz + deterministic variants	Passed; O(1) ceiling dust per fresh tranche
INV-07	System rebalance moves deviation toward target without creating value	Controller, Balancer	Fuzz + boundary tests	Passed
INV-08	Fees are bounded, taken once, and protocol earmarks remain backed	FeePolicy, Controller	Fuzz + settlement tests	Passed under owner parameters satisfying setters
INV-09	Flash mint/redeem round trips are restorative absent declared fees	Controller	Metamorphic tests	Passed
INV-10	Reserve tranche count stays within enforced maximum	PerpTranche	Boundary + fuzz	Passed at 50-token limit
INV-11	Vault note mint/burn preserves proportional ownership	Both vaults	Multi-depositor lifecycle tests	Passed for enumerated TVL assets; LOW-02 describes the intentional upkeep-dependent exit
INV-12	First depositor cannot publicly front-run initialization	Both vaults	Access + minimum-liquidity tests	Passed; seeding is owner-only
INV-13	Deposit/redeem loops cannot create value absent market movement	Vaults and routers	Round-trip and multi-user tests	Passed under standard tokens and zero/declared fees
INV-14	Reentrant token callbacks cannot reenter guarded vault/controller flows	Controller and vaults	ReentrantToken tests	Passed for guarded entries; routers are deliberately unguarded and non-custodial
INV-15	DEX execution cannot settle below the vault's oracle-derived floor	USD vault, strategies	Malicious strategy + boundary tests	Passed; external oracle and liquidity remain trusted
INV-16	CoW orders bind receiver, pair, direction, value floor, deadline, and flags	CoW broker	Unit + integrated malicious-order tests	Passed
INV-17	Flash-loan callbacks accept only pinned router/lender and repay atomically	CoW broker	Callback and short/overloan tests	Passed under trusted lender semantics
INV-18	Pausing blocks intended surfaces without seizing balances	Controller and vaults	Access matrix tests	Passed; availability remains keeper-dependent
INV-19	Current oracle reads reject zero/stale/incomplete/L2-down states	Oracle wrappers	Unit tests + manual trace	Passed; heartbeat correctness is deployment-specific
INV-20	Routers forward only value attributable to the current call	Routers	Donation/allowance tests	Failed for ambient balances; LOW-01

Findings Summary

ID	Title	Severity	Confidence	Status	Component
LOW-01	Stateless routers expose ambient balances and persistent approvals	Low	High	Acknowledged / unresolved	Core and vault routers
LOW-02	Redemption without upkeep leaves the caller's resting-underlying share behind	Low	High	Acknowledged design	UsdPerpSrVault
INFO-01	Upgrade and configuration authority has protocol-wide impact	Informational	High	Design risk	ProxyAdmin and owners
INFO-02	Accounting assumes standard ERC-20 behavior	Informational	High	Design risk	Token integrations
INFO-03	Permissionless maintenance safety depends on owner parameters	Informational	High	Design risk	UnderlyingPerpVault
INFO-04	Permissionless settlement uses the first poke's post-eligibility price	Informational	High	Acknowledged design	BondsManager

Detailed Findings

INFO-04 (formerly MED-01) — Permissionless settlement uses the first poke's post-eligibility price

Severity: Informational. Confidence: High. Status: Acknowledged design. Category: economic state transition and oracle timing.

Affected code: `core-contracts/contracts/bonds/BondsManager.sol:310-330`; evidence at `core-contracts/test/bonds/BondsManager.maturity.ts:271-341`.

`mature` becomes callable by any account after the bond timestamp. It reads the current oracle price in the settlement transaction and irrevocably stores the resulting senior and junior claims. There is no historical price lookup, bounded observation window, or deterministic rule selecting an observation associated with the maturity timestamp. Until one transaction succeeds, view functions also continue simulating the split at each new live price.

The original Medium assessment assumed that a dated instrument's payout should be tied to an observation at its scheduled maturity boundary. The developers clarified that this is not the intended invariant: `maturityDate` is the earliest date after which the bond can be matured, and the current price when the required poke executes is intentionally the settlement price. Interface comments support that interpretation by describing the field as the timestamp when `mature()` becomes callable.

Preconditions are a passed timestamp, an unsettled bond, and a changed valid oracle price. A senior or junior beneficiary, keeper, or MEV actor waits for or races on a favorable price and calls `mature`. No capital or special privilege is required beyond gas and ordering cost. In the repository's deterministic fixture, a 100-unit 50/50 bond pays 50/50 at the opening price, 25/75 after a doubling, and 100/0 after a crash. Total collateral remains solvent, but the entire junior claim can be transferred to the senior side or most senior entitlement can move to juniors. The blast radius is one bond series and all its holders; claims cannot be recomputed after settlement.

Developer response: “Maturity date is the date after which the bond can be mature, not a fixed point in time. The contract has to be poked for maturity to execute, and that determines the settlement price. That is the design.”

Auditor resolution: The response invalidates the fixed-boundary invariant used for the original severity, but not the demonstrated behavior. Both tranche classes have opposing incentives over the first post-eligibility poke, any account can submit it, and ordering fixes the observation irreversibly. Because this is an intended instrument property rather than a violation of its specified rules, the finding is downgraded from Medium to Informational. User-facing specifications should call the timestamp a settlement-eligibility boundary, state that claims continue repricing until the first successful poke, describe ordering/MEV effects, and document any automation or monitoring expectations. A historical-price or bounded-window design remains an optional hardening choice, not required remediation for conformance to the stated design.

LOW-02 (formerly MED-02) — Redemption without upkeep leaves the caller's resting-underlying share behind

Severity: Low. Confidence: High. Status: Acknowledged design. Category: share accounting and asset enumeration.

Affected code: `vaults/contracts/UsdPerpSrVault.sol:989-1020,1033-1037`; router mitigation at `vaults/contracts/routers/UsdPerpSrVaultRouter.sol:121-130`; evidence at `vaults/test/UsdPerpSrVaultRouter.ts:371-410`.

The vault can retain underlying after passive settlement because it sizes a flash redemption for the maximum accepted fee but pays the exact promised output. The code intentionally haircuts newly minted notes for this resting value. In contrast, `getTVL` and `computeRedemptionAmts` enumerate only USD and perpSr. A direct redemption burns notes without returning or pricing the holder's proportional underlying.

The original Medium assessment treated proportional ownership over every vault-controlled asset as an unconditional redemption invariant. The developers clarified that users are expected and economically incentivized to call upkeep before redeeming. The router exposes this choice explicitly, its documentation says that upkeep prices the caller's resting-underlying share in, and repository tests state that the UI enables the flag.

Developer response: “Users are naturally incentivized to call upkeep on redeem.”

Auditor resolution: The incentive and disclosed router flag substantially reduce likelihood; no attacker can force an informed holder to burn notes through the discounted path. The behavior nevertheless remains loss-inducing when callers use direct `redeem`, disable the flag, or need liquidity while upkeep is unavailable because of minimum-size, pause, DEX-health, USD-health, or oracle-health gates. Once notes are burned, the omitted share benefits remaining holders and cannot be reclaimed. The finding is therefore downgraded from Medium to Low. Recommended hardening is to make upkeep the safe default, expose the value omitted by a direct redemption in previews and interfaces, warn integrators clearly, and consider returning proportional underlying when conversion is unavailable. If all production interfaces enforce or prominently disclose this behavior, Informational classification would also be reasonable.

LOW-01 — Stateless routers expose ambient balances and persistent approvals to arbitrary callers and prior spenders

Severity: Low. Confidence: High. Status: Acknowledged in code; unresolved. Category: token custody and approvals.

Affected code includes `core-contracts/contracts/Router.sol:203-204,462-469`, `vaults/contracts/_utils/RouterHelpers.sol:19-44`, and the public entry points in `PerpVaultRouter`, `UnderlyingPerpVaultRouter`, and `UsdPerpSrVaultRouter`.

Several router exits forward the contract's entire token balance instead of a call-local delta. Zero-amount calls therefore transfer donated or stranded tokens to an unrelated caller. Separately, caller-selected controllers and strategies receive maximum allowances that persist after the call; an earlier spender can later pull tokens that arrive at the router. Core `Router.redeem3` similarly reports and forwards its complete underlying balance.

The broken property is call isolation for ambient router state. Preconditions are a mistaken direct transfer, stranded balance, or later transfer to a previously approved token/spender pair. Capturing the balance costs one transaction and is irreversible, but normal successful routes end cleanly and the NatSpec explicitly says the routers are not custody. Protocol vault/controller balances are not directly affected, so severity is Low despite a potentially high damage-to-gas ratio for an integration mistake.

Reproduce with LOW-01 commands in `proofs-of-concept/README.md`. Measure before/after balances and forward only operation deltas; reject or recover unexpected pre-existing balances to a fixed address; validate vault/controller/strategy relationships; and revoke allowances granted to caller-selected spenders. Regression tests should pre-donate every routed token and assert that unrelated zero-value callers cannot receive it.

Economic and Design Risks

Privileged authority

The proxy architecture does not enforce a timelock or multisig. A ProxyAdmin owner can deploy arbitrary implementation logic. Component owners can replace the issuer, fee policy, balancer, oracles, DEX strategy, passive swappers, fee caps, target bands, liquidity thresholds, and maintenance timing. These are intended privileges, not unprivileged exploits, but compromise can cause protocol-wide loss or freeze. Production governance was not supplied and must be verified independently.

Token assumptions

Bonds and perps account the requested transfer amount rather than a realized balance delta and explicitly assume non-deflationary, non-rebasing assets. Vault and strategy correctness also assumes standard transfer semantics, bounded decimals, and no hostile callback/accounting behavior. Governance whitelisting is the main control. Treat token upgrades, blacklisting, pausing, and issuer keys as protocol dependencies.

Oracle, liquidity, and liveness

Oracle freshness checks do not guarantee economic correctness, historical observations, market liquidity, or resistance to a compromised reporter. DEX health and reference-price gates can deliberately halt maintenance in adverse markets. Redemptions remain possible in several outage states, but LOW-02 makes the safe exit path dependent on asset enumeration and upkeep availability. Keepers must monitor settlement, issuance, rollover, fee settlement, upkeep, passive inventory, and pool health.

Permissionless maintenance

Permissionless execution improves liveness but turns owner parameters into economic security controls. Existing tests show that zero minimum rebalance size, short cadence, and permissive fee cap allow repeated small fee-bearing rebalances. Use net-benefit thresholds, cumulative loss budgets, and alerting rather than relying only on per-call bounds.

MEV and ordering

User operations include minimum outputs, but users must set meaningful values. Bond settlement has no analogous user-selected price constraint and is directly exposed to transaction ordering. Passive CoW orders are live-oracle bound at validation, limiting stale price fills but making them sensitive to oracle movement and solver timing.

Centralization and Privileged Operations

Capability	Role	Maximum code-level impact	Recommended control
Upgrade all transparent proxies	ProxyAdmin owner	Replace logic; move, freeze, mint, burn, or corrupt all proxied state	Timelocked multisig; independent review; monitored upgrade simulation
Replace issuer, fee policy, balancer	Controller owner	Change issuance and economic execution; halt or distort mint/redeem/rebalance	Timelock, bounded adapters, staged rollout
Replace/clear oracles	BondsManager/controller owner	Misprice or halt bonds, perps, and USD vault	Feed allowlist, delay, dual-source monitoring
Add/remove bond assets and issuers	BondsManager owner	List incompatible asset or authorize series creation	Asset committee, behavior checklist, cap and delay
Change fee curves, protocol fee, AUM fee	FeePolicy owner	Charge up to setter bounds; redirect protocol fee collection	Published caps, timelock, monitoring
Reassign perp controller	Perp token owner	New controller can move reserves and mint/burn	Same governance as upgrade authority
Pause user/system/vault flows	Keeper	Temporary denial of service; no direct seizure in current logic	Separate emergency multisig, expiry and incident policy
Change controller value cap	Keeper	Permit or prevent additional value entry	Bound change rate; monitoring
Change vault strategy and approvals	Vault owner	Grant unlimited token allowances to a strategy	Curated strategy registry, timelock, allowance monitor
Allow passive swappers	USD vault owner	Permit swaps within floor/cap/session gates	Solver/broker allowlist process and revocation monitoring
Change vault fee/cadence/size/health parameters	Vault owner	Allow loss within caps or halt maintenance	Conservative hard bounds and delayed changes
Recover arbitrary tokens from perps	Controller through perp	Move non-reserve/mature reserve assets according to controller logic	Invariant monitoring; no arbitrary recipient path

No source-level user-fund seizure or arbitrary owner withdrawal function was identified in the reviewed implementations. Upgrade authority can introduce one and therefore dominates the trust model.

Testing Results

Baseline

The first root compile failed because the shell exposed Node 16.14.2. Re-running with the repository's Node 22.15.0 runtime succeeded. Hardhat emitted a support warning for Solidity 0.8.35, Solidity emitted future-keyword warnings in OpenZeppelin sources and a transient-storage composability warning, and no error prevented compilation.

The root test command passed 1,638 core tests, 838 vault tests, subgraph Matchstick tests, and Go tests. Root lint passed ESLint, Solhint, Prettier checks, Go vet, and module verification. Non-fatal registry update checks could not resolve `registry.npmjs.org` in the restricted network environment.

Fuzz and invariant campaign

`FUZZ_RUNS=50 PATH_FUZZ_RUNS=30 FUZZ_SEED=202608281359` ran 81 repository-native fuzz/invariant test cases in approximately two minutes. The suites use the full requested count or source-defined fractions/caps for expensive properties. They covered bond settlement bounds and isolation, fee curves/caps, controller mint/redeem and slippage, reserve limits, rollover conservation, system backing, random operation sequences, and helper differential formulas. No failure was retained.

Coverage

The core non-fork suite reported 100% statements, 93.92% branches, 100% functions, and 99.73% lines. The vault non-fork suite reported 100% statements, 96.75% branches, 100% functions, and 99.87% lines. Interfaces, proxy import sentinels, and test mocks were excluded by instrumentation. Fork-only live DEX tests were not run because no RPC/fork scope was supplied.

High coverage is not treated as proof of safety. INFO-04, LOW-02, and LOW-01 are themselves demonstrated by passing tests, illustrating that coverage measures execution rather than specification or integration safety.

Negative results

Attempts to turn callback-capable router tokens into a completed in-flight theft were invalidated by atomic outer-call failure in the modeled path. CoW underpriced orders, forged fields, foreign receivers, wrong directions, relayer allowance abuse, and loan callback misuse reverted. Vault and controller reentrancy guards held. First-depositor manipulation was blocked by owner-only seeding and locked minimum liquidity. Rollover rounding did not produce repeatable extractable value. Fee settlement stayed backed. These results apply only to the tested models.

Informational Observations

1. Replace `maxSellAmt * qMid >= minBuyAmt * mid` in `BaseDexStrategy.maxFillable` with a full-precision ratio comparison to avoid theoretical overflow at extreme supplies.
2. Enforce and document the PerpBalancer tranche-count assumption locally even though current perp storage caps the list at 50.
3. Reject zero addresses for critical owner/keeper/dependency setters unless disabling the component is an explicit, tested emergency operation.
4. Emit complete old/new values for all economic configuration changes and operate event-driven alerts.
5. Publish units and rounding direction beside every externally configurable amount; raw-token and 1e18-value fields coexist throughout vault code.
6. Add a production-configuration test that loads deployed addresses and asserts owner, ProxyAdmin, timelock, feed, heartbeat, strategy, pool, allowance, fee, cadence, and cap values.
7. Preserve deterministic fuzz seeds in CI and add a nightly deeper corpus with sequence-depth and total transition counters.
8. Treat comments that explicitly acknowledge a loss or public sweep as a prompt for API-level guardrails, not only integration documentation.

Conclusion

The reviewed commit shows strong defensive engineering: extensive tests, high instrumented coverage, initialization locks, broad slippage and oracle checks, transient reentrancy guards on custody contracts, bounded reserve arrays, carefully pinned CoW callbacks, and explicit economic invariants. Those controls prevented the direct theft, insolvency, fee-accounting, reentrancy, and order-forgery paths exercised during the audit.

After developer response, no Critical, High, or Medium issue remains. INFO-04 records the intended first-poke settlement semantics and the need to disclose their timing and ordering consequences. LOW-02 records the avoidable but irreversible value left behind by redemption without successful upkeep. LOW-01 records ambient router balance and allowance exposure. These behaviors are deterministic; the revised classifications reflect design intent, user control, and the absence of an attacker-forced loss path.

Residual risk remains concentrated in upgrade keys, component owners, configured tokens, live oracle quality, DEX liquidity, keeper liveness, and economic parameter selection. A remediation review should retest the exact proofs, adjacent paths, storage/migration implications, and all conservation/share invariants. This report does not assert that untested vulnerabilities are absent.

Appendix A — Complete File Inventory

Every supplied Solidity source file and supporting architecture artifact reviewed during this engagement is listed below. Test-only contracts are evidence rather than deployable scope.

Path	Classification	Status
core-contracts/contracts/PerpController.sol	production implementation	Reviewed
core-contracts/contracts/Router.sol	production implementation	Reviewed
core-contracts/contracts/_external/ProxyImports.sol	dependency compile sentinel	Reviewed
core-contracts/contracts/_interfaces/Constants.sol	interface / type definition	Reviewed
core-contracts/contracts/_interfaces/Errors.sol	interface / type definition	Reviewed
core-contracts/contracts/_interfaces/IBondIssuer.sol	interface / type definition	Reviewed
core-contracts/contracts/_interfaces/IBondsManager.sol	interface / type definition	Reviewed
core-contracts/contracts/_interfaces/IChainlinkOracle.sol	interface / type definition	Reviewed
core-contracts/contracts/_interfaces/IERC20Burnable.sol	interface / type definition	Reviewed
core-contracts/contracts/_interfaces/IFeePolicy.sol	interface / type definition	Reviewed
core-contracts/contracts/_interfaces/IOwnable.sol	interface / type definition	Reviewed
core-contracts/contracts/_interfaces/IPerp.sol	interface / type definition	Reviewed
core-contracts/contracts/_interfaces/IPerpBalancer.sol	interface / type definition	Reviewed
core-contracts/contracts/_interfaces/IPerpController.sol	interface / type definition	Reviewed
core-contracts/contracts/_interfaces/IPriceOracle.sol	interface / type definition	Reviewed
core-contracts/contracts/_interfaces/ITranche.sol	interface / type definition	Reviewed
core-contracts/contracts/_interfaces/Types.sol	interface / type definition	Reviewed
core-contracts/contracts/_test/CachedOracleTester.sol	test / malicious mock	Reviewed
core-contracts/contracts/_test/DMock.sol	test / malicious mock	Reviewed
core-contracts/contracts/_test/ERC20HelpersTester.sol	test / malicious mock	Reviewed
core-contracts/contracts/_test/LineHelpersTester.sol	test / malicious mock	Reviewed
core-contracts/contracts/_test/MathHelpersTester.sol	test / malicious mock	Reviewed
core-contracts/contracts/_test/MockBondManager.sol	test / malicious mock	Reviewed
core-contracts/contracts/_test/MockChainlinkOracle.sol	test / malicious mock	Reviewed
core-contracts/contracts/_test/MockERC20.sol	test / malicious mock	Reviewed
core-contracts/contracts/_test/MockOracle.sol	test / malicious mock	Reviewed
core-contracts/contracts/_test/OracleCacheTester.sol	test / malicious mock	Reviewed
core-contracts/contracts/_test/OwnedMintableERC20.sol	test / malicious mock	Reviewed
core-contracts/contracts/_test/PerpBalancerTester.sol	test / malicious mock	Reviewed
core-contracts/contracts/_test/PerpHelpersTester.sol	test / malicious mock	Reviewed
core-contracts/contracts/_test/PerpSupplyMock.sol	test / malicious mock	Reviewed
core-contracts/contracts/_test/ShrinkTester.sol	test / malicious mock	Reviewed
core-contracts/contracts/_test/SlotDerivationTester.sol	test / malicious mock	Reviewed
core-contracts/contracts/_test/TrancheDataMock.sol	test / malicious mock	Reviewed
core-contracts/contracts/_test/TrancheHelpersTester.sol	test / malicious mock	Reviewed
core-contracts/contracts/_utils/ERC20Helpers.sol	production library	Reviewed
core-contracts/contracts/_utils/LineHelpers.sol	production library	Reviewed
core-contracts/contracts/_utils/MathHelpers.sol	production library	Reviewed
core-contracts/contracts/_utils/PerpBalancerHelpers.sol	production library	Reviewed
core-contracts/contracts/_utils/PerpHelpers.sol	production library	Reviewed
core-contracts/contracts/_utils/Shrink.sol	production library	Reviewed
core-contracts/contracts/_utils/TrancheHelpers.sol	production library	Reviewed

Path	Classification	Status
core-contracts/contracts/bonds/BondsManager.sol	production implementation	Reviewed
core-contracts/contracts/bonds/Tranche.sol	production implementation	Reviewed
core-contracts/contracts/oracles/CLOracleWrapper.sol	production implementation	Reviewed
core-contracts/contracts/oracles/CachedOracle.sol	production implementation	Reviewed
core-contracts/contracts/oracles/L2CLOracleWrapper.sol	production implementation	Reviewed
core-contracts/contracts/perps/PerpJr.sol	production implementation	Reviewed
core-contracts/contracts/perps/PerpSr.sol	production implementation	Reviewed
core-contracts/contracts/perps/PerpTranche.sol	production implementation	Reviewed
core-contracts/contracts/strategies/BondIssuer.sol	production implementation	Reviewed
core-contracts/contracts/strategies/FeePolicy.sol	production implementation	Reviewed
core-contracts/contracts/strategies/PerpBalancer.sol	production implementation	Reviewed
vaults/contracts/UnderlyingPerpVault.sol	production implementation	Reviewed
vaults/contracts/UsdPerpSrVault.sol	production implementation	Reviewed
vaults/contracts/_external/ProxyImports.sol	dependency compile sentinel	Reviewed
vaults/contracts/_interfaces/CommonErrors.sol	interface / type definition	Reviewed
vaults/contracts/_interfaces/GPv2Order.sol	interface / type definition	Reviewed
vaults/contracts/_interfaces/IComposableCoW.sol	interface / type definition	Reviewed
vaults/contracts/_interfaces/IConditionalOrder.sol	interface / type definition	Reviewed
vaults/contracts/_interfaces/ICowFlashLoanRouter.sol	interface / type definition	Reviewed
vaults/contracts/_interfaces/ICurveStableSwapNG.sol	interface / type definition	Reviewed
vaults/contracts/_interfaces/ICurveTwoCryptoNG.sol	interface / type definition	Reviewed
vaults/contracts/_interfaces/IDexStrategy.sol	interface / type definition	Reviewed
vaults/contracts/_interfaces/IGPv2Settlement.sol	interface / type definition	Reviewed
vaults/contracts/_interfaces/IMorpho.sol	interface / type definition	Reviewed
vaults/contracts/_interfaces/Types.sol	interface / type definition	Reviewed
vaults/contracts/_test/MockBondIssuer.sol	test / malicious mock	Reviewed
vaults/contracts/_test/MockBondsManager.sol	test / malicious mock	Reviewed
vaults/contracts/_test/MockCow.sol	test / malicious mock	Reviewed
vaults/contracts/_test/MockCurveStableSwapNG.sol	test / malicious mock	Reviewed
vaults/contracts/_test/MockCurveTwoCryptoNG.sol	test / malicious mock	Reviewed
vaults/contracts/_test/MockDexStrategy.sol	test / malicious mock	Reviewed
vaults/contracts/_test/MockERC20.sol	test / malicious mock	Reviewed
vaults/contracts/_test/MockMorphoBorrower.sol	test / malicious mock	Reviewed
vaults/contracts/_test/MockPerp.sol	test / malicious mock	Reviewed
vaults/contracts/_test/MockPerpController.sol	test / malicious mock	Reviewed
vaults/contracts/_test/MockPriceOracle.sol	test / malicious mock	Reviewed
vaults/contracts/_test/MockTranche.sol	test / malicious mock	Reviewed
vaults/contracts/_test/MockUniswapV4PoolManager.sol	test / malicious mock	Reviewed
vaults/contracts/_test/PerpVaultHelpersTester.sol	test / malicious mock	Reviewed
vaults/contracts/_test/ReentrantToken.sol	test / malicious mock	Reviewed
vaults/contracts/_test/TestnetOracleStrategy.sol	test / malicious mock	Reviewed
vaults/contracts/_test/_IntegrationAggregator.sol	test / malicious mock	Reviewed
vaults/contracts/_utils/PerpVaultHelpers.sol	production library	Reviewed
vaults/contracts/_utils/RouterHelpers.sol	production library	Reviewed
vaults/contracts/routers/PerpVaultRouter.sol	production implementation	Reviewed
vaults/contracts/routers/UnderlyingPerpVaultRouter.sol	production implementation	Reviewed

Path	Classification	Status
vaults/contracts/routers/UsdPerpSrVaultCowBroker.sol	production implementation	Reviewed
vaults/contracts/routers/UsdPerpSrVaultRouter.sol	production implementation	Reviewed
vaults/contracts/strategies/BaseDexStrategy.sol	production implementation	Reviewed
vaults/contracts/strategies/CurveCbBtcUsdcStrategy.sol	production implementation	Reviewed
vaults/contracts/strategies/UniswapV4CbBtcUsdcStrategy.sol	production implementation	Reviewed
core-contracts/ignition/modules/TwoFactorSepolia.ts	supporting architecture / configuration / prior assessment	Reviewed as supporting material
vaults/ignition/modules/VaultsSepolia.ts	supporting architecture / configuration / prior assessment	Reviewed as supporting material
core-contracts/hardhat.config.ts	supporting architecture / configuration / prior assessment	Reviewed as supporting material
vaults/hardhat.config.ts	supporting architecture / configuration / prior assessment	Reviewed as supporting material
core-contracts/package.json	supporting architecture / configuration / prior assessment	Reviewed as supporting material
vaults/package.json	supporting architecture / configuration / prior assessment	Reviewed as supporting material
package.json	supporting architecture / configuration / prior assessment	Reviewed as supporting material
README.md	supporting architecture / configuration / prior assessment	Reviewed as supporting material
core-contracts/README.md	supporting architecture / configuration / prior assessment	Reviewed as supporting material
vaults/README.md	supporting architecture / configuration / prior assessment	Reviewed as supporting material
.audits/PeckShield-Audit-Report-2Factor-v1.0.pdf	supporting architecture / configuration / prior assessment	Reviewed as supporting material

Appendix B — Public Function and Role Inventory

Public and external implementation functions are listed. Access labels are mechanically extracted and manually checked at the role level; complete internal/private rows are in the delivered CSV.

Path:line	Contract.function	Visibility	Access surface
core-contracts/contracts/PerpController.sol:237	PerpController.init	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/PerpController.sol:265	PerpController.updateIssuer	external	owner
core-contracts/contracts/PerpController.sol:275	PerpController.updateFeePolicy	external	owner
core-contracts/contracts/PerpController.sol:281	PerpController.updatePerpBalancer	external	owner
core-contracts/contracts/PerpController.sol:287	PerpController.updateRolloverEligibility	external	owner
core-contracts/contracts/PerpController.sol:302	PerpController.updateDRTargets	external	owner
core-contracts/contracts/PerpController.sol:323	PerpController.updateMinCleanupVal	external	owner
core-contracts/contracts/PerpController.sol:333	PerpController.updateLiquidityThresholds	external	owner
core-contracts/contracts/PerpController.sol:348	PerpController.updateRebalanceWindow	external	owner
core-contracts/contracts/PerpController.sol:366	PerpController.setUnderlyingOracle	external	owner
core-contracts/contracts/PerpController.sol:371	PerpController.resetUnderlyingOracle	external	owner
core-contracts/contracts/PerpController.sol:377	PerpController.updateKeeper	external	owner
core-contracts/contracts/PerpController.sol:386	PerpController.pauseSystemOps	external	keeper
core-contracts/contracts/PerpController.sol:392	PerpController.pauseUserOps	external	keeper
core-contracts/contracts/PerpController.sol:399	PerpController.updateValueCap	external	keeper
core-contracts/contracts/PerpController.sol:410	PerpController.pullProtocolFees	external	protocol fee collector
core-contracts/contracts/PerpController.sol:423	PerpController.rebalance	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/PerpController.sol:447	PerpController.rollover	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/PerpController.sol:465	PerpController.upkeep	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/PerpController.sol:483	PerpController.mint2	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/PerpController.sol:574	PerpController.redeem2	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/PerpController.sol:675	PerpController.flashMint2	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/PerpController.sol:795	PerpController.flashRedeem2	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/PerpController.sol:886	PerpController.perpSr	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/PerpController.sol:891	PerpController.perpJr	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/PerpController.sol:896	PerpController.latestBondInfo	external	unrestricted / caller-constrained by function checks

Path:line	Contract.function	Visibility	Access surface
core-contracts/contracts/PerpController.sol:906	PerpController.systemDeviations	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/PerpController.sol:919	PerpController.computeDRs	public	unrestricted / caller-constrained by function checks
core-contracts/contracts/PerpController.sol:937	PerpController.systemReserveState	public	unrestricted / caller-constrained by function checks
core-contracts/contracts/PerpController.sol:953	PerpController.reservedPerpJrLiquidity	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/PerpController.sol:959	PerpController.availPerpJrLiquidity	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/PerpController.sol:971	PerpController.rebalanceWindow	public	unrestricted / caller-constrained by function checks
core-contracts/contracts/PerpController.sol:985	PerpController.collectedFeeBal	public	unrestricted / caller-constrained by function checks
core-contracts/contracts/Router.sol:71	Router.trancheAndMint	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/Router.sol:159	Router.redeem3	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/Router.sol:241	Router.mint2	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/Router.sol:296	Router.redeem2	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/Router.sol:351	Router.flashMint2	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/Router.sol:396	Router.flashRedeem2	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/BondsManager.sol:139	BondsManager.init	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/BondsManager.sol:151	BondsManager.addIssuer	external	owner
core-contracts/contracts/bonds/BondsManager.sol:159	BondsManager.removeIssuer	external	owner
core-contracts/contracts/bonds/BondsManager.sol:167	BondsManager.addAsset	external	owner
core-contracts/contracts/bonds/BondsManager.sol:175	BondsManager.removeAsset	external	owner
core-contracts/contracts/bonds/BondsManager.sol:183	BondsManager.updateOracle	external	owner
core-contracts/contracts/bonds/BondsManager.sol:199	BondsManager.createBond	external	authorized issuer
core-contracts/contracts/bonds/BondsManager.sol:262	BondsManager.deposit	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/BondsManager.sol:301	BondsManager.tryMature	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/BondsManager.sol:310	BondsManager.mature	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/BondsManager.sol:336	BondsManager.redeemAll	external	unrestricted / caller-constrained by function checks

Path:line	Contract.function	Visibility	Access surface
core-contracts/contracts/bonds/BondsManager.sol:372	BondsManager.redeem	public	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/BondsManager.sol:405	BondsManager.redeemMature	public	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/BondsManager.sol:456	BondsManager.redeemImmature	public	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/BondsManager.sol:519	BondsManager.assetCount	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/BondsManager.sol:526	BondsManager.assetAt	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/BondsManager.sol:533	BondsManager.oracleOf	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/BondsManager.sol:540	BondsManager.assetPrice	public	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/BondsManager.sol:552	BondsManager.isValidBond	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/BondsManager.sol:557	BondsManager.data	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/BondsManager.sol:564	BondsManager.underlying	public	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/BondsManager.sol:569	BondsManager.isMature	public	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/BondsManager.sol:574	BondsManager.maturityDate	public	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/BondsManager.sol:579	BondsManager.secondsToMaturity	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/BondsManager.sol:590	BondsManager.srTranche	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/BondsManager.sol:595	BondsManager.jrTranche	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/BondsManager.sol:600	BondsManager.tranches	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/BondsManager.sol:608	BondsManager.bondClaim	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/BondsManager.sol:617	BondsManager.isValidTranche	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/BondsManager.sol:629	BondsManager.trancheData	public	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/BondsManager.sol:703	BondsManager.computeSettlement	public	unrestricted / caller-constrained by function checks

Path:line	Contract.function	Visibility	Access surface
core-contracts/contracts/bonds/BondsManager.sol:727	BondsManager.computeBondID	public	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/Tranche.sol:47	Tranche.init	public	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/Tranche.sol:89	Tranche.mint	external	owner
core-contracts/contracts/bonds/Tranche.sol:95	Tranche.burn	external	owner
core-contracts/contracts/bonds/Tranche.sol:100	Tranche.manager	public	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/Tranche.sol:105	Tranche.data	public	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/Tranche.sol:110	Tranche.symbol	public	unrestricted / caller-constrained by function checks
core-contracts/contracts/bonds/Tranche.sol:115	Tranche.name	public	unrestricted / caller-constrained by function checks
core-contracts/contracts/oracles/CachedOracle.sol:26	CachedOracle.getValidPrice	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/oracles/CachedOracle.sol:41	CachedOracle.cachePrice	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/oracles/CachedOracle.sol:48	CachedOracle.resetCache	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/perps/PerpJr.sol:31	PerpJr.init	external	configured controller
core-contracts/contracts/perps/PerpJr.sol:42	PerpJr.primeApproval	external	configured controller
core-contracts/contracts/perps/PerpJr.sol:48	PerpJr.revokeApproval	external	configured controller
core-contracts/contracts/perps/PerpJr.sol:58	PerpJr.bondDepositsForRollover	external	configured controller
core-contracts/contracts/perps/PerpSr.sol:23	PerpSr.init	public	unrestricted / caller-constrained by function checks
core-contracts/contracts/perps/PerpTranche.sol:106	PerpTranche.updateController	external	owner
core-contracts/contracts/perps/PerpTranche.sol:114	PerpTranche.mintTo	external	configured controller
core-contracts/contracts/perps/PerpTranche.sol:119	PerpTranche.burnFrom	external	configured controller
core-contracts/contracts/perps/PerpTranche.sol:127	PerpTranche.transferReservesTo	public	configured controller
core-contracts/contracts/perps/PerpTranche.sol:148	PerpTranche.handleTransferReservesIn	public	configured controller
core-contracts/contracts/perps/PerpTranche.sol:165	PerpTranche.registerTranche	external	configured controller
core-contracts/contracts/perps/PerpTranche.sol:182	PerpTranche.logBalance	external	configured controller
core-contracts/contracts/perps/PerpTranche.sol:192	PerpTranche.recover	external	configured controller
core-contracts/contracts/perps/PerpTranche.sol:219	PerpTranche.reserves	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/perps/PerpTranche.sol:241	PerpTranche.reserveData	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/perps/PerpTranche.sol:265	PerpTranche.underlying	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/perps/PerpTranche.sol:269	PerpTranche.controller	external	unrestricted / caller-constrained by function checks

Path:line	Contract.function	Visibility	Access surface
core-contracts/contracts/perps/PerpTranche.sol:275	PerpTranche.maxTrancheCount	public	unrestricted / caller-constrained by function checks
core-contracts/contracts/strategies/BondIssuer.sol:66	BondIssuer.init	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/strategies/BondIssuer.sol:82	BondIssuer.updateBondConfig	external	owner
core-contracts/contracts/strategies/BondIssuer.sol:96	BondIssuer.updateIssuanceTimingConfig	public	owner
core-contracts/contracts/strategies/BondIssuer.sol:115	BondIssuer.issue	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/strategies/BondIssuer.sol:170	BondIssuer.cleanupMature	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/strategies/BondIssuer.sol:187	BondIssuer.activeBondIDs	public	unrestricted / caller-constrained by function checks
core-contracts/contracts/strategies/BondIssuer.sol:195	BondIssuer.latestBondInfo	public	unrestricted / caller-constrained by function checks
core-contracts/contracts/strategies/FeePolicy.sol:119	FeePolicy.init	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/strategies/FeePolicy.sol:189	FeePolicy.updateFeeParams	public	owner
core-contracts/contracts/strategies/FeePolicy.sol:202	FeePolicy.updateRebalanceParams	public	owner
core-contracts/contracts/strategies/FeePolicy.sol:215	FeePolicy.updateProtocolFeeConfig	external	owner
core-contracts/contracts/strategies/FeePolicy.sol:229	FeePolicy.updateAumFeePerc	external	owner
core-contracts/contracts/strategies/FeePolicy.sol:250	FeePolicy.computeOpFees	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/strategies/FeePolicy.sol:285	FeePolicy.computeRebalanceVal	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/strategies/PerpBalancer.sol:110	PerpBalancer.computeSystemRebalancePlan	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/strategies/PerpBalancer.sol:154	PerpBalancer.computePerpSrRolloverPlan	external	unrestricted / caller-constrained by function checks
core-contracts/contracts/strategies/PerpBalancer.sol:222	PerpBalancer.computePerpSrRolloverState	public	unrestricted / caller-constrained by function checks
core-contracts/contracts/strategies/PerpBalancer.sol:284	PerpBalancer.computePerpJrLiquidity	public	unrestricted / caller-constrained by function checks
core-contracts/contracts/strategies/PerpBalancer.sol:375	PerpBalancer.computeRolloverPolicy	public	unrestricted / caller-constrained by function checks
core-contracts/contracts/strategies/PerpBalancer.sol:508	PerpBalancer.computeBackupRolloverPolicy	public	unrestricted / caller-constrained by function checks
vaults/contracts/UnderlyingPerpVault.sol:204	UnderlyingPerpVault.init	public	unrestricted / caller-constrained by function checks
vaults/contracts/UnderlyingPerpVault.sol:246	UnderlyingPerpVault.updateKeeper	public	owner
vaults/contracts/UnderlyingPerpVault.sol:254	UnderlyingPerpVault.updateTargetDR	public	owner
vaults/contracts/UnderlyingPerpVault.sol:268	UnderlyingPerpVault.updateMinRebalanceAmt	public	owner

Path:line	Contract.function	Visibility	Access surface
vaults/contracts/UnderlyingPerpVault.sol:275	UnderlyingPerpVault.updateRebalanceTimingParams	public	owner
vaults/contracts/UnderlyingPerpVault.sol:286	UnderlyingPerpVault.updateMaxFlashFeePerc	external	owner
vaults/contracts/UnderlyingPerpVault.sol:295	UnderlyingPerpVault.pause	external	keeper
vaults/contracts/UnderlyingPerpVault.sol:300	UnderlyingPerpVault.unpause	external	keeper
vaults/contracts/UnderlyingPerpVault.sol:311	UnderlyingPerpVault.initDeposit	external	owner
vaults/contracts/UnderlyingPerpVault.sol:339	UnderlyingPerpVault.deposit	external	unrestricted / caller-constrained by function checks
vaults/contracts/UnderlyingPerpVault.sol:374	UnderlyingPerpVault.redeem	external	unrestricted / caller-constrained by function checks
vaults/contracts/UnderlyingPerpVault.sol:402	UnderlyingPerpVault.rebalance	external	unrestricted / caller-constrained by function checks
vaults/contracts/UnderlyingPerpVault.sol:459	UnderlyingPerpVault.computeMintAmt	public	unrestricted / caller-constrained by function checks
vaults/contracts/UnderlyingPerpVault.sol:499	UnderlyingPerpVault.computeRedemptionAmts	public	unrestricted / caller-constrained by function checks
vaults/contracts/UnderlyingPerpVault.sol:516	UnderlyingPerpVault.getTVL	public	unrestricted / caller-constrained by function checks
vaults/contracts/UnderlyingPerpVault.sol:534	UnderlyingPerpVault.computeRebalanceAmount	public	unrestricted / caller-constrained by function checks
vaults/contracts/UnderlyingPerpVault.sol:553	UnderlyingPerpVault.underlyingBalance	external	unrestricted / caller-constrained by function checks
vaults/contracts/UnderlyingPerpVault.sol:558	UnderlyingPerpVault.perpBalance	external	unrestricted / caller-constrained by function checks
vaults/contracts/UsdPerpSrVault.sol:325	UsdPerpSrVault.init	external	unrestricted / caller-constrained by function checks
vaults/contracts/UsdPerpSrVault.sol:376	UsdPerpSrVault.updateKeeper	public	owner
vaults/contracts/UsdPerpSrVault.sol:383	UsdPerpSrVault.updateSwapper	public	owner
vaults/contracts/UsdPerpSrVault.sol:391	UsdPerpSrVault.updateTargetDR	public	owner
vaults/contracts/UsdPerpSrVault.sol:407	UsdPerpSrVault.updateOracleDeviationPerc	public	owner
vaults/contracts/UsdPerpSrVault.sol:421	UsdPerpSrVault.updateDexStrategy	public	owner
vaults/contracts/UsdPerpSrVault.sol:441	UsdPerpSrVault.updateOpLimits	public	owner
vaults/contracts/UsdPerpSrVault.sol:465	UsdPerpSrVault.updateSessionParams	public	owner
vaults/contracts/UsdPerpSrVault.sol:497	UsdPerpSrVault.updateHealthCheckParams	public	owner
vaults/contracts/UsdPerpSrVault.sol:517	UsdPerpSrVault.pause	external	keeper
vaults/contracts/UsdPerpSrVault.sol:522	UsdPerpSrVault.unpause	external	keeper
vaults/contracts/UsdPerpSrVault.sol:534	UsdPerpSrVault.initDeposit	external	owner
vaults/contracts/UsdPerpSrVault.sol:562	UsdPerpSrVault.deposit	external	unrestricted / caller-constrained by function checks
vaults/contracts/UsdPerpSrVault.sol:602	UsdPerpSrVault.redeem	external	unrestricted / caller-constrained by function checks

Path:line	Contract.function	Visibility	Access surface
vaults/contracts/UsdPerpSrVault.sol:633	UsdPerpSrVault.rebalance	external	unrestricted / caller-constrained by function checks
vaults/contracts/UsdPerpSrVault.sol:682	UsdPerpSrVault.swap	external	unrestricted / caller-constrained by function checks
vaults/contracts/UsdPerpSrVault.sol:738	UsdPerpSrVault.upkeep	external	unrestricted / caller-constrained by function checks
vaults/contracts/UsdPerpSrVault.sol:809	UsdPerpSrVault.rebalanceCapacity	public	unrestricted / caller-constrained by function checks
vaults/contracts/UsdPerpSrVault.sol:848	UsdPerpSrVault.sessionCapacity	public	unrestricted / caller-constrained by function checks
vaults/contracts/UsdPerpSrVault.sol:882	UsdPerpSrVault.maxAcceptableTerms	external	unrestricted / caller-constrained by function checks
vaults/contracts/UsdPerpSrVault.sol:941	UsdPerpSrVault.isAcceptable	external	unrestricted / caller-constrained by function checks
vaults/contracts/UsdPerpSrVault.sol:955	UsdPerpSrVault.computeMintAmt	public	unrestricted / caller-constrained by function checks
vaults/contracts/UsdPerpSrVault.sol:1004	UsdPerpSrVault.computeRedemptionAmts	public	unrestricted / caller-constrained by function checks
vaults/contracts/UsdPerpSrVault.sol:1015	UsdPerpSrVault.getTVL	public	unrestricted / caller-constrained by function checks
vaults/contracts/UsdPerpSrVault.sol:1024	UsdPerpSrVault.usdBalance	public	unrestricted / caller-constrained by function checks
vaults/contracts/UsdPerpSrVault.sol:1029	UsdPerpSrVault.perpSrBalance	public	unrestricted / caller-constrained by function checks
vaults/contracts/UsdPerpSrVault.sol:1036	UsdPerpSrVault.underlyingBalance	public	unrestricted / caller-constrained by function checks
vaults/contracts/routers/PerpVaultRouter.sol:39	PerpVaultRouter.swapUsdToPerp	external	unrestricted / caller-constrained by function checks
vaults/contracts/routers/PerpVaultRouter.sol:83	PerpVaultRouter.swapPerpToUsd	external	unrestricted / caller-constrained by function checks
vaults/contracts/routers/UnderlyingPerpVaultRouter.sol:44	UnderlyingPerpVaultRouter.depositUnderlying	external	unrestricted / caller-constrained by function checks
vaults/contracts/routers/UnderlyingPerpVaultRouter.sol:103	UnderlyingPerpVaultRouter.redeemUnderlying	external	unrestricted / caller-constrained by function checks
vaults/contracts/routers/UsdPerpSrVaultCowBroker.sol:238	UsdPerpSrVaultCowBroker.settleToVault	public	unrestricted / caller-constrained by function checks
vaults/contracts/routers/UsdPerpSrVaultCowBroker.sol:251	UsdPerpSrVaultCowBroker.flashLoanAndCallBack	external	unrestricted / caller-constrained by function checks
vaults/contracts/routers/UsdPerpSrVaultCowBroker.sol:271	UsdPerpSrVaultCowBroker.onMorphoFlashLoan	external	unrestricted / caller-constrained by function checks
vaults/contracts/routers/UsdPerpSrVaultCowBroker.sol:289	UsdPerpSrVaultCowBroker.provideInventory	external	pinned trampoline

Path:line	Contract.function	Visibility	Access surface
vaults/contracts/routers/UsdPerpSrVaultCowBroker.sol:307	UsdPerpSrVaultCowBroker.isValidSignature	external	unrestricted / caller-constrained by function checks
vaults/contracts/routers/UsdPerpSrVaultCowBroker.sol:323	UsdPerpSrVaultCowBroker.getTradeableOrder	external	unrestricted / caller-constrained by function checks
vaults/contracts/routers/UsdPerpSrVaultCowBroker.sol:342	UsdPerpSrVaultCowBroker.verify	external	unrestricted / caller-constrained by function checks
vaults/contracts/routers/UsdPerpSrVaultRouter.sol:54	UsdPerpSrVaultRouter.depositUsd	external	unrestricted / caller-constrained by function checks
vaults/contracts/routers/UsdPerpSrVaultRouter.sol:121	UsdPerpSrVaultRouter.redeemUsd	external	unrestricted / caller-constrained by function checks
vaults/contracts/strategies/BaseDexStrategy.sol:25	BaseDexStrategy.quote	external	unrestricted / caller-constrained by function checks
vaults/contracts/strategies/BaseDexStrategy.sol:32	BaseDexStrategy.trade	external	unrestricted / caller-constrained by function checks
vaults/contracts/strategies/BaseDexStrategy.sol:39	BaseDexStrategy.referencePrice	external	unrestricted / caller-constrained by function checks
vaults/contracts/strategies/BaseDexStrategy.sol:45	BaseDexStrategy.isHealthy	external	unrestricted / caller-constrained by function checks
vaults/contracts/strategies/BaseDexStrategy.sol:51	BaseDexStrategy.maxFillable	public	unrestricted / caller-constrained by function checks
vaults/contracts/strategies/CurveCbBtcUsdcStrategy.sol:123	CurveCbBtcUsdcStrategy.quote	public	unrestricted / caller-constrained by function checks
vaults/contracts/strategies/CurveCbBtcUsdcStrategy.sol:147	CurveCbBtcUsdcStrategy.trade	external	unrestricted / caller-constrained by function checks
vaults/contracts/strategies/CurveCbBtcUsdcStrategy.sol:201	CurveCbBtcUsdcStrategy.referencePrice	public	unrestricted / caller-constrained by function checks
vaults/contracts/strategies/CurveCbBtcUsdcStrategy.sol:223	CurveCbBtcUsdcStrategy.isHealthy	external	unrestricted / caller-constrained by function checks
vaults/contracts/strategies/UniswapV4CbBtcUsdcStrategy.sol:156	UniswapV4CbBtcUsdcStrategy.quote	public	unrestricted / caller-constrained by function checks
vaults/contracts/strategies/UniswapV4CbBtcUsdcStrategy.sol:184	UniswapV4CbBtcUsdcStrategy.trade	external	unrestricted / caller-constrained by function checks
vaults/contracts/strategies/UniswapV4CbBtcUsdcStrategy.sol:206	UniswapV4CbBtcUsdcStrategy.unlockCallback	external	unrestricted / caller-constrained by function checks
vaults/contracts/strategies/UniswapV4CbBtcUsdcStrategy.sol:258	UniswapV4CbBtcUsdcStrategy.referencePrice	public	unrestricted / caller-constrained by function checks
vaults/contracts/strategies/UniswapV4CbBtcUsdcStrategy.sol:288	UniswapV4CbBtcUsdcStrategy.isHealthy	external	unrestricted / caller-constrained by function checks

Appendix C — Candidate Disposition

Each retained hypothesis was challenged before confirmation or dismissal.

Candidate	Disposition	Basis
Settlement price can be selected after scheduled maturity	Acknowledged design risk	The mechanism and payout divergence are confirmed. Developer response establishes that the timestamp is intentionally an eligibility boundary and the first poke's current price intentionally settles the bond, invalidating the fixed-boundary invariant behind the original Medium severity.
Resting underlying is omitted from note redemption	Acknowledged design risk	The deterministic value redistribution is confirmed. Developer response establishes upkeep as the intended, economically incentivized safe path; optional/direct redemption and upkeep liveness constraints retain a Low-severity integration risk.
Router ambient balances and allowances are public value surfaces	Confirmed	Deterministic zero-call sweep and retained-allowance tests; limited to router balances.
Router callback reentrancy steals an in-flight standard flow	Invalidated	The existing test does not arm a callback; attempted nested sweeps leave the outer operation underfunded and revert atomically under the modeled flow. Callback tokens remain excluded/hardening territory.
PerpController fee bookkeeping permits protocol fee undercollateralization	Invalidated	Fee balance reconciliation, cap logic, pull limits, and settlement tests preserve backing.
PerpBalancer rollover creates value through rounding	Invalidated	Ceiling dust is bounded to O(1) units per fresh tranche; deterministic and fuzz rollover conservation properties passed.
PerpBalancer uint8 loops can wrap on unbounded tranche arrays	Invalidated for current system	PerpTranche enforces a 50-tranche reserve limit, below uint8 capacity. The balancer's standalone 255-tranche assumption should remain documented.
BaseDexStrategy cross multiplication overflows in normal deployments	Informational	Overflow requires extreme products far outside intended cbBTC/USDC balances; governance-curated assets and practical supplies constrain reachability, but mulDiv would harden the comparison.
CoW broker standing relayer allowances permit underpriced inventory drain	Invalidated	Pinned receiver, pair, direction, expiry, FOK flags, zero fee, ERC-1271 hash binding, and live oracle floor were exercised; underpriced settlement reverted atomically.
Uniswap V4 quote optimism causes silent vault loss	Invalidated	Vault-side realized-rate floor reverts the complete transaction when execution returns too little; remaining risk is liveness and external pool/oracle trust.
First depositor can inflate vault share price	Invalidated	Owner-only seed and locked minimum liquidity prevent public seed front-running in tested vaults.
Permissionless small rebalances can erode value	Design risk	Reachable only within owner-configured timing, size, and fee tolerances.
Single administrator can replace logic and price sources	Design risk	Transparent proxy and owner capability; production governance was not supplied for verification.
Nonstandard ERC-20 behavior desynchronizes accounting	Design risk	Explicit standard-token assumptions and amount-based accounting; asset curation is the primary control.
BondIssuer active-set cleanup becomes permanently uncallable	Invalidated for bounded intended configuration	Normal active set is bounded by issue interval and maturity; the same price source is required for issuance and maturation, limiting adversarial accumulation. Monitor configuration and oracle outages.
Cached oracle transient storage can be poisoned by an external caller	Invalidated	The cache stores only the oracle's own current valid price and is transaction-local; controller/vault wrappers reset it around state-changing flows.

Appendix D — Tools and Commands

Tool	Version / configuration	Purpose
Node.js	22.15.0 (repo runtime); initial shell 16.14.2	Build and TypeScript tests
Yarn	4.9.1	Workspace orchestration
Hardhat	2.29.0	Compile, local EVM, tests
Solidity	0.8.35; optimizer 800; via-IR; Cancun	Contract compilation
Solhint	6.2.3	Solidity lint
Solidity Coverage	0.8.17	Statement, branch, function, line coverage
OpenZeppelin Contracts	5.6.1	Proxy, token, access, math dependencies
Go	go1.26.7 invoked by repository Makefile	Operations build/test
ReportLab	4.4.9	PDF generation
Poppler	runtime-provided	PDF render/text validation

```
PATH=/Users/nithin/.nvm/versions/node/v22.15.0/bin:$PATH yarn compile

PATH=/Users/nithin/.nvm/versions/node/v22.15.0/bin:$PATH yarn test

PATH=/Users/nithin/.nvm/versions/node/v22.15.0/bin:$PATH yarn lint

FUZZ_RUNS=50 PATH_FUZZ_RUNS=30 FUZZ_SEED=202608281359 PATH=/Users/nithin/.nvm/versions/node/v22.15.0/bin:$PATH
yarn workspace @2ff/core-contracts test:fuzz

(cd core-contracts && FUZZ_RUNS=0 PATH_FUZZ_RUNS=0 yarn hardhat coverage)

(cd vaults && yarn hardhat coverage --testfiles 'test/*.ts')
```

Appendix E — Glossary and Disclaimer

CDR / DR. Collateralization or deviation ratio describing senior/junior reserve balance.
Perp. Reserve-bearing senior or junior perpetual ERC-20.
Tranche. Dated senior or junior bond claim.
Resting underlying. Underlying held directly by the USD vault between settlement and upkeep.
Ambient balance. Router token balance outside the current call's measured delta.
FOK. Fill-or-kill order.

This assessment is a time-bounded review of the identified commit under stated assumptions. It is not a warranty, guarantee, financial recommendation, or substitute for secure governance, deployment review, monitoring, incident response, and continued testing. Smart contracts and dependencies can fail through code, configuration, operations, markets, governance, or infrastructure. Remediation should receive an independent review.